High Performance Computing – PGAS and UPC Exercise

Orazgali Azamat, BDA-2402

The programs were compiled and run with Berkeley UPC 2022.10.0.

I had to use Docker and .bat files since -C++ (g++ 4.9.2) and other Windows compilers cannot compile .upc files, and Berkeley UPC does not run natively on Windows. I built Berkeley UPC from its official source inside a Linux Docker container. The docker command is long, so each program has its own .bat. Each one builds the image the first time, then compiles and runs the program with 4 threads. The executable is created inside the container, so no build files are left in the folder.

Part 1: Understanding the PGAS Model

1. What does PGAS stand for?
Partitioned Global Address Space.

2. What is meant by a global address space?
One address space that all threads can read and write directly by variable name or array index, even if the memory is physically spread over different processors or nodes.

3. Why is the global address space described as partitioned?
It is divided into parts, one per thread. Each part is stored in the local memory of its thread.

4. What is meant by the affinity of a shared data item?
The thread in whose partition (local memory) the item is stored.

5. What is a local memory access?
An access by a thread to its private data or to shared data that has affinity to that thread. It is fast.

6. What is a non-local memory access?
An access to shared data that has affinity to another thread. It may need communication and is slower.

7. Can a UPC thread access shared data that has affinity to another thread?
Yes. It simply reads or writes the shared variable, and the runtime does the transfer.

8. What important difference exists between non-local access in MPI and non-local access in a PGAS model?
In MPI the data must be moved with explicit messages: the owner sends and the other

process receives. In PGAS a thread reads or writes the remote data directly with normal syntax, and the owner thread does not take part.

Part 2: Shared and Private Variables

1. How many copies of x exist?
One.

2. Which thread has affinity to x?
Thread 0.

3. How many elements does y contain?
4 (THREADS).

4. Which thread has affinity to each element of y?
y[i] has affinity to thread i: y[0] to T0, y[1] to T1, y[2] to T2, y[3] to T3.

5. How many copies of z exist?
Four, one in each thread.

6. Is z shared between the four threads?
No. It is private; each thread has its own copy.

Variable	Shared / Private	Affinity
x	Shared	Thread 0
y[0]	Shared	Thread 0
y[1]	Shared	Thread 1
y[2]	Shared	Thread 2
y[3]	Shared	Thread 3
z	Private	Each thread has its own copy

Part 3: Blocking and Affinity

thread(i) = floor(i / 4) mod 4. I also checked the table with upc_threadof() on shared [4] int A[32] using 4 threads, and the result was the same.

Array Elements	Thread with Affinity
A[0]–A[3]	0
A[4]–A[7]	1
A[8]–A[11]	2
A[12]–A[15]	3
A[16]–A[19]	0
A[20]–A[23]	1
A[24]–A[27]	2

A[28]–A[31]	3

1. Which thread has affinity to A[2]?
Thread 0 (floor(2/4) = 0, 0 mod 4 = 0).

2. Which thread has affinity to A[6]?
Thread 1 (floor(6/4) = 1).

3. Which thread has affinity to A[13]?
Thread 3 (floor(13/4) = 3).

4. Which thread has affinity to A[22]?
Thread 1 (floor(22/4) = 5, 5 mod 4 = 1).

5. Why might the choice of block size affect program performance?
The block size decides which elements are local to each thread. If a thread works on neighbouring elements, a bigger block keeps them on the same thread, so there are fewer non-local accesses. A block size that does not match the access pattern causes many remote accesses. A block that is too big can give unequal work, with some threads getting more elements or none at all.

Part 4: First UPC Program

File: hello_upc.upc, run with run_hello_upc.bat (4 threads).

Thread	Printed Message
0	Hello from thread 0 of 4
1	Hello from thread 1 of 4
2	Hello from thread 2 of 4
3	Hello from thread 3 of 4

Order of the lines in three runs: run 1: 3, 0, 2, 1; run 2: 0, 2, 1, 3; run 3: 0, 2, 1, 3.

1. What does MYTHREAD represent?
The index of the current thread, from 0 to THREADS - 1.

2. What does THREADS represent?
The total number of UPC threads in the program (4 here).

3. Does every UPC thread execute the same program?
Yes. UPC uses the SPMD model: all threads run the same code with different MYTHREAD values.

4. Is the order of the printed messages guaranteed?
No. The threads run independently, and the order changed between runs.

5. In what way is this execution model similar to MPI?
Both are SPMD. A fixed number of copies of the program is started by a launcher (upcrun / mpiexec). MYTHREAD is like the rank from MPI_Comm_rank and THREADS is like the size from MPI_Comm_size.

Part 5: Parallel Vector Addition

File: vector_add.upc, run with run_vector_add.bat. The blank was filled with C[i] = A[i] + B[i];

Output:
C[0] = 0
C[10] = 30
C[999] = 2997

This matches C[i] = 3i: 3·0 = 0, 3·10 = 30, 3·999 = 2997.

1. Can different values of Ci be calculated independently?
Yes. C[i] depends only on A[i] and B[i], not on other elements.

2. What is the purpose of upc_forall?
It splits the loop iterations between the threads, so each thread runs only its own iterations.

3. What does the fourth expression of upc_forall specify?
The affinity expression, which decides which thread runs iteration i. With &C[i], the thread that has affinity to C[i] runs it. With an integer, the thread number is that value mod THREADS.

4. Why is distributing work according to data affinity potentially useful?
The thread works on data stored in its own memory, so the accesses are local and need no communication. A, B and C have the same layout, so A[i], B[i] and C[i] are all local to the thread that computes C[i].

5. Does every thread calculate all N elements?
No. Each thread calculates only the elements with affinity to it, about N/THREADS = 250 each (i mod 4 = MYTHREAD).

Part 6: Observing Data Affinity

File: affinity.upc, run with run_affinity.bat. The program printed all 16 elements; A[8]–A[15] repeat the same pattern (A[8] thread 0, A[9] thread 1, ..., A[15] thread 3).

Element	Thread with Affinity
A[0]	0
A[1]	1
A[2]	2
A[3]	3
A[4]	0
A[5]	1
A[6]	2
A[7]	3

1. What is the default block size of a UPC shared array?
1, so the elements are dealt to the threads one by one (cyclic).

2. Does the distribution repeat after all threads receive one block?
Yes. After A[3] goes to thread 3, A[4] goes to thread 0 again, and so on round-robin.

3. Would accessing an element having affinity to MYTHREAD normally be considered local or non-local?
Local.

Part 7: Local and Non-Local Access

Thread 0:

Access	Local / Non-local
A[0]	Local
A[1]	Non-local
A[2]	Non-local
A[3]	Non-local

Thread 2:

Access	Local / Non-local
A[0]	Non-local
A[1]	Non-local
A[2]	Local
A[3]	Non-local

1. Why can excessive non-local accesses reduce performance?
Each non-local access has higher latency and may need a network transfer. With many of them, threads spend their time waiting for data instead of computing.

2. Why should work preferably be assigned to a thread that has affinity to the data being processed?
Then most accesses are local and fast, and communication is kept to a minimum (the owner computes).

3. How is this idea related to data locality discussed earlier in the course?
It is the same principle as cache locality with row-major access (week 2) and moving computation to the data in MapReduce/HDFS (week 6). Data that is close to the processor is cheaper to access, so the computation should run where the data is.

Part 8: Synchronization

File: synchronization.upc, run with run_synchronization.bat. upc_barrier; was added after the write.

Output:
result[0] = 0
result[1] = 10
result[2] = 20
result[3] = 30

1. Why is a barrier required before thread 0 reads all results?
Thread 0 must wait until every thread has written its result[MYTHREAD]. Without the barrier it could read an element before it is written.

2. What does a thread do when it reaches upc_barrier before the other threads?
It waits there until all threads have reached the barrier.

3. What potential problem could occur if the barrier were removed?
A race condition: thread 0 could print 0 or an old value for threads that have not written yet, and the output could change between runs.

4. Does UPC automatically synchronize threads after every shared-memory operation?
No. The programmer must add synchronization with barriers, locks or fences.

Part 9: Shared Pointers

Pointer	Pointer Location	Pointed-to Data
p1	private	private
p2	private	shared
p3	shared	private
p4	shared	shared

1. Which pointer type can be used to access arbitrary locations in the shared address space?
A pointer-to-shared (shared int *), i.e. p2 or p4.

2. What is meant by a pointer-to-shared?
A pointer that points into the shared address space. It holds the address, the thread with affinity and the position in the block, so it can point to shared data on any thread. Pointer arithmetic follows the array's block layout.

3. What locations may an ordinary C pointer safely reference in the UPC memory model?
The thread's own private data, and shared data with affinity to the same thread (after casting it to a normal pointer). It should not reference shared data of other threads.

Part 10: Dynamic Shared Memory

1. Which operation is collective?
upc_all_alloc().

2. Which operation is normally expected to be called by one thread?
upc_global_alloc().

3. When upc_all_alloc() is called, do all threads receive the same shared pointer?
Yes, all threads get the same pointer.

4. What function is used to free dynamically allocated shared memory?
upc_free() (UPC 1.3 also has the collective upc_all_free()).

Part 11: Comparing Programming Models

Model	Multiple Memories	Non-local Access	Explicit Messages
OpenMP	No (one shared memory)	No (all memory is accessed the same way)	No
MPI	Yes	Only through messages	Yes
PGAS / UPC	Yes (partitioned)	Yes, directly by reading/writing shared data	No

1. What feature of PGAS makes it look similar to shared-memory programming?
The global address space: any thread can read and write any shared variable with normal syntax, without messages.

2. What feature of PGAS makes data locality still important?
The partitioning: every shared item has affinity to one thread, and non-local accesses are slower than local ones.

3. In MPI, how would a process normally obtain data stored by another process?
With explicit messages: the owner calls MPI_Send and the process calls MPI_Recv, or they use a collective such as MPI_Bcast or MPI_Scatter.

4. In UPC, how can a thread obtain shared data having affinity to another thread?
By reading the shared variable or array element directly (or through a pointer-to-shared or upc_memget for a block).

5. Why can PGAS be viewed as a model between traditional shared-memory and message-passing programming?
It is programmed like shared memory (one address space, no messages), but the memory is distributed as in MPI, and the programmer still has to care about where data is stored.

Part 12: Short Discussion

1. What is data affinity in PGAS?
The thread whose local part of the shared memory holds a shared data item.

2. What is the difference between shared and private data in UPC?
Shared data exists once in the global space and all threads can access it. Private data exists separately in every thread, and only that thread can access it.

3. What is the purpose of a blocked shared array?
To control how the elements are distributed, keeping groups of consecutive elements on the same thread to match the access pattern and improve locality.

4. What is the purpose of upc_forall?
To share loop iterations between threads according to an affinity expression, so each thread works mostly on its own data.

5. Why are local accesses generally preferable to non-local accesses?
They use the thread's own memory, so they have lower latency and need no communication.

6. What is the purpose of upc_barrier?
To make all threads wait until every thread reaches it, so shared writes made before the barrier are complete and visible after it.

7. What is the difference between a blocking barrier and a split-phase barrier?
upc_barrier blocks in one step until all threads arrive. A split-phase barrier uses

upc_notify to signal arrival and upc_wait to wait. Between the two calls the thread can do local work, so waiting overlaps with computation.

8. What problem can locks solve when several threads write to the same shared data?
Race conditions and lost updates in read-modify-write operations. A lock (upc_lock/upc_unlock) lets only one thread update the data at a time.

9. What is one major conceptual difference between UPC and MPI?
UPC has one global address space and threads access remote data directly. MPI processes have separate memories and communicate only with explicit messages.

10. What is one major conceptual difference between UPC and OpenMP?
OpenMP assumes one shared memory where all accesses cost the same and has no notion of affinity. UPC exposes data affinity (local vs non-local) and can run on distributed-memory clusters.